

身份证实名核验服务（sfsm）• API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。

版本：sfsm（身份证实名核验） | 通信：HTTP + JSON | 编码：UTF-8

本服务采用统一请求信封（`appKey` / `sign` / `encryptionType` / `body`）与统一响应结构（`head` + `body`）。调用方对 `body` 业务参数按 MD5 规则加签即可。

本接口入参仅 `name` 与 `idCard` 两项且均必填，不需要手机号。 若额外传入其它字段（如 `mobile`），本服务不将其用于查询，但仍会参与加签（见 2.3）。

「一致」与「不一致」都属于查得。 只要本服务给出了明确的核验结论，`body.code` 即为 001（计费）；姓名与身份证号是否一致，请读 `body.result.range` 里的 `result` 与 `desc` 字段，**不要用 `body.code` 判断一致性。** `body.code` = 999 仅表示查无记录。详见 3.1.5 与 4.2。

核验结果为结构化对象，本服务将其序列化为 JSON 字符串置于 `body.result.range`，调用方需二次解析（见 3.1.5）。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「身份证实名核验服务（sfsm）」的第三方产品技术开发人员与日常运维人员。

1.2 接入须知

- 正式服务地址（任选其一，两域名等价）：`http://aiszcloud.cn:8080`、`http://aiszcloud.com.cn:8080`。
- 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。

1.3 接口说明

项目	说明
请求方式	POST（查得数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ≥ 10 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 <code>head.errorCode</code> / <code>body.code</code> 表达
签名	调用时需对 <code>body</code> 中的业务参数 + 我方分配的 <code>appSecret</code> 进行 MD5 加签，详见第二章

1.4 环境说明

- **正式环境**：使用正式账户，调用已开通接口，按查得成功条数计费（见第五章）。
- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y890sfsm	String	是	我方分配给客户的公开标识
sign	7388c24a8546864b6c6589c86e8ee816	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要额外的产品编号参数（产品由请求路径区分）。 appKey / sign / encryptionType 不参与签名计算。
调用方传入的 body 字段一律为明文（姓名/身份证号）。

2.2 鉴权校验顺序

本服务按以下顺序校验，任一失败立即返回对应 head.errorCode （参数非法直接拒绝，不计费）：

1. appKey 是否存在 → 否则 505001
2. appKey 是否匹配到账户 → 否则 505004
3. 账户是否有效（启用且在有效期内） → 否则 505007
4. 签名是否正确 → 否则 505002

2.3 加签方式

1. 取出 body 中所有**非空**的业务参数（不含文件/字节流类型，不含值为空的参数）。
2. 按参数名（key）的 **ASCII 升序**排序；首字符相同则依次比较后续字符。
3. 将排序后的参数按 参数名 参数值 直接拼接，最后追加 appSecret ，得到待签名串。
4. 对待签名串做 **MD5**，取 32 位**小写**十六进制字符串，赋值给 sign 。

示例: `body = { "idCard": "11010519491231002X", "name": "张三" }`, `appSecret = "<你的密钥>"`。

按 ASCII 升序排序后 key 顺序为 `idCard → name`, 待签名串为:

```
idCard11010519491231002Xname张三<你的密钥>
```

`sign = MD5(待签名串)` 的小写十六进制值。

提示: 拼接顺序由 key 的 ASCII 决定, 请勿写死字段顺序。加签的范围是你实际发送的 `body` 全部非空字段——若额外传了本接口不需要的字段, 它同样要参与加签, 否则会被判 `505002`。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exception {
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()):  # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue  # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest()  # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 身份证实名核验 (sfsfm)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxSFSM
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxSFSM (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxSFSM)
鉴权	appKey + MD5 签名 (见第二章)

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
name	张三	String	是	姓名，明文传入
idCard	11010519491231002X	String	是	身份证号（18 位，末位可为 x ），须为校验位合法的有效号码，明文传入

本接口不需要手机号。 只有上表两个字段参与查询。

name 缺失、idCard 缺失或格式非法（非 18 位规范号码）将返回 head.errorCode = 505062，直接拒绝，不计费。

身份证号仅满足 18 位格式但校验位不正确时，也可能返回 505062（服务处理异常），请使用真实有效的身份证号联调。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y890sfsm",
  "sign": "<按 2.3 规则计算>",
  "body": {
    "name": "张三",
    "idCard": "11010519491231002X"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时请提供
time	948	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得核验结论（一致与不一致均属此列）【计费】；999 = 查无记录【不计费】
msg	成功	String	业务描述
reqid	1kf9x2ac	String	本次请求流水号
uid	a1b2c3d4-e5f6a7b8	String	交易流水号（对账用），与 head.logId 同值
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	"{...}"	String	核验结果的 JSON 字符串，需二次解析，结构见 3.1.5

3.1.4 响应示例

① 查得核验结论 —— 一致（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4-e5f6a7b8", "time": 948, "timestamp": 1718456789012 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "1kf9x2ab",
    "uid": "a1b2c3d4-e5f6a7b8",
    "result": {
      "range": "{\"result\":0,\"desc\":\"一致\",\"sex\":\"女\",\"birthday\":\"194***31\", \"address\":\"北京市西城区\"}"
    }
  }
}
```

② 查得核验结论 —— 不一致（同样计费， body.code 仍为 001 ）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4-c9d0e1f2", "time": 873, "timestamp": 1718456789015 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "lkf9x2ad",
    "uid": "a1b2c3d4-c9d0e1f2",
    "result": {
      "range": "{\\"result\\":1,\\"desc\\":\\"不一致\\",\\"sex\\":\\"女\\",\\"birthday\\":\\"194***31\\",\\"address\\":\\"北京市西城区\\"}"
    }
  }
}
```

③ 查无记录（不计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 96, "timestamp": 1718456789013 },
  "body": {
    "code": "999",
    "msg": "没有查询到数据",
    "reqid": "lkf9x2ac",
    "uid": "a1b2c3d5"
  }
}
```

④ 网关级错误（无 body）

```
{
  "head": { "errorCode": "505062", "errorMsg": "数据请求异常", "logId": "a1b2c3d6", "time": 12, "timestamp": 1718456789014 }
}
```

3.1.5 result.range 结构说明

`result.range` 是核验结果序列化后的 **JSON 字符串**。调用方应先对该字符串做一次 `JSON.parse` / 反序列化，解析结果为一个 JSON 对象，可直接读取以下字段：

字段	类型	说明
result	Number	核验结论：0 = 姓名与身份证号一致；1 = 不一致
desc	String	核验结论的文字描述，如 一致 / 不一致
sex	String	性别
birthday	String	生日
address	String	籍贯

判定一致性请只看 `result`（或 `desc`），不要看 `body.code`。`body.code = 001` 表示本服务成功取得了核验结论，一致与不一致都在其中；只有查无记录才返回 `body.code = 999`（此时无 `result` 节点）。

`sex / birthday / address` 为附带的辅助信息，可能按脱敏规则部分打码（如 `194***31`），仅供人工比对参考，请勿用于精确匹配。

说明：`result.range` 的具体字段以实际返回为准，本服务保证原样透出。建议调用方按「存在即解析、缺失即忽略」做容错处理。

解析示例 (JavaScript)：

```
const row = JSON.parse(resp.body.result.range);
if (resp.body.code === "001") {
  if (row.result === 0) {
    console.log("一致", row.desc);
  } else {
    console.log("不一致", row.desc);
  }
}
```

3.2 成功查得数查询（扩展接口）

查询本账户累计成功查得数据的次数与累计调用次数，用于客户侧自助监控。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	GET /v1/openapi/zlx/quotaSFSM
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/quotaSFSM（或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaSFSM）
鉴权	同 3.1 主查询（请求体携带 <code>appKey</code> + <code>sign</code> 信封； <code>body</code> 可为 <code>{}</code> ，此时 <code>sign = MD5(appSecret)</code> ）

响应示例

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1350
}
```

参数	说明
status	账户状态（ACTIVE/SUSPENDED 等）
serviceUsed	累计成功查得数据的次数（仅统计查得成功 code=001 ）
totalCalls	累计调用次数（含查得、查无及已完成的查询请求）

说明：无任何额度上限拦截，仅做成功查得数与调用次数统计。

3.3 健康检查

项目	内容
路径	GET /healthz
完整地址	http://aiszcloud.cn:8080/healthz （或 http://aiszcloud.com.cn:8080/healthz ）
鉴权	无
响应	HTTP 200，纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务侧的账户/参数/系统等异常均归一为网关 505062，不计费，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	是否计费
001	查得核验结论（range.result 为 0 一致或 1 不一致）	计费
999	查无记录	不计费

注意：001 只代表「本服务取得了明确的核验结论」，不代表「一致」。一致与不一致同为查得、同样计费，区分请读 range.result / range.desc（见 3.1.5）。

五、计费说明

- 仅当返回 body.code = 001（查得核验结论）时，才计入成功查得数并对客户计费——无论结论是一致还是不一致。
- body.code = 999（查无记录）**不计费**。
- 网关级错误（head.errorCode 非 0：鉴权失败、参数非法、服务处理异常、系统异常等）**一律不计费**。
- 计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 appKey、appSecret；服务地址见 1.2 接入须知。

- 2. 按第二章实现加签，先在测试环境联调，再切正式环境。
- 3. 实现结论解析：先看 `body.code`（001 有结论 / 999 无记录），再从 `result.range` 解析 `result` 判定一致性。
- 4. 上线后通过成功查得数查询接口（3.2）监控调用量。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（≥ 10 秒）。
- 收到网络超时/无响应时可安全重试：本服务基于内部流水号做幂等，**不会因重试重复计费**。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 505062、鉴权错误 505001 / 505002 / 505004），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 appKey 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）；确认实际发送的每一个 body 字段都参与了加签
505007	联系商务确认账户状态与有效期
505062	检查 name 与 idCard 是否齐全且身份证号有效（含校验位）；若入参正常仍持续出现，记录 logId 联系我方
拿到 001 却以为「一致」	001 仅表示有结论；一致性看 <code>result.range</code> 的 <code>result</code> 字段（见 3.1.5、4.2）

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- ☐ 服务地址（`aiszcloud.cn:8080` 或 `aiszcloud.com.cn:8080`）、`appKey`、`appSecret`、环境匹配无误
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ `name` 与 `idCard` 均以明文传入且都不为空
- ☐ 使用校验位合法的真实身份证号（勿用随意编造的 18 位号码）
- ☐ 清楚本接口**不需要手机号**；若仍传其它字段，已把它们计入加签范围
- ☐ 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- ☐ 已实现「001 不等于一致」的判定逻辑：从 `result.range` 读 `result` 字段判定一致性
- ☐ 已实现对 `result.range` 字符串的二次解析与字段容错
- ☐ 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 <code>sign</code> ，切勿泄露或随请求传输
logId	全链路追踪 ID (= <code>head.logId</code>)，排障/对账唯一凭据
uid	交易流水号 (= <code>body.uid</code>)，对账用，与 <code>logId</code> 同值
range	核验结果的 JSON 字符串，需二次解析（结构见 3.1.5）